

Distributed Systems

Lecture12: Code Migration

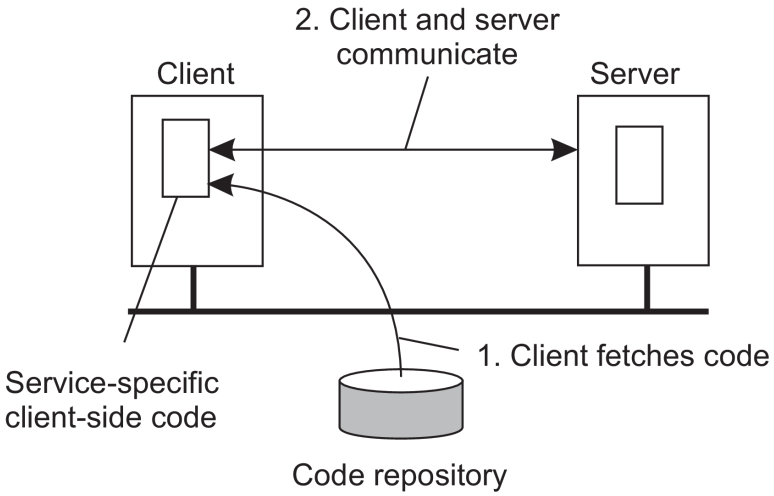
Mohammad Reza Shakournia

shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

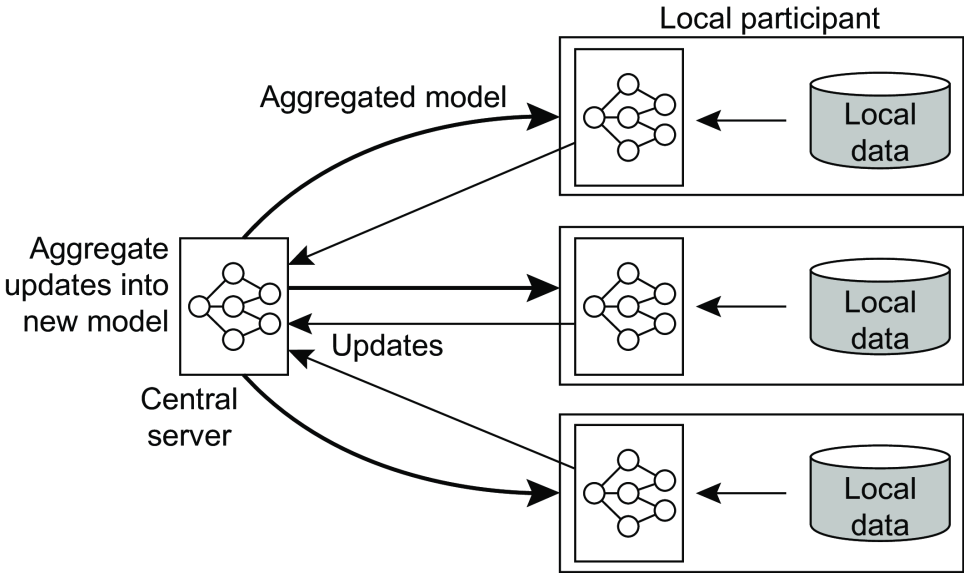
Reasons to Migrate Code

- **Load distribution**:
  - Ensuring that servers in a data center are sufficiently loaded (e.g., to prevent waste of energy)
  - Minimizing communication by ensuring that computations are close to where the data is (think of mobile computing).
- **Flexibility: moving code to a client when needed**

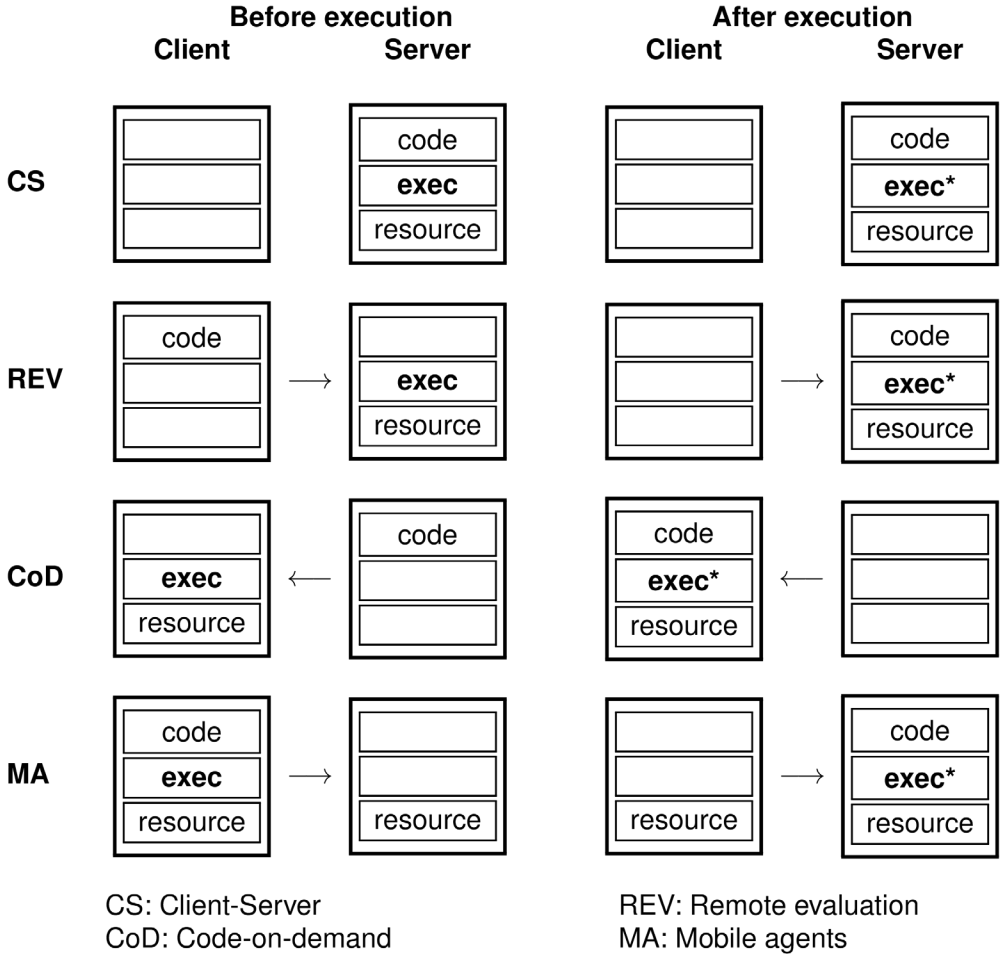


Reasons to Migrate Code

- **Privacy and security**:
  - In many cases, one cannot move data to another location, for whatever reason (often legal ones). Solution: move the code to the data.
- **Example: federated machine learning**



Paradigms for Code Mobility



Strong and Weak Mobility

- **Object components**
  - Code segment: contains the actual code
  - Data segment: contains the state
  - Execution state: contains context of thread executing the object's code
- **Weak mobility:** Move only code and data segment (and reboot execution)
  - Relatively simple, especially if code is portable
  - Distinguish code shipping (push) from code fetching (pull)
- **Strong mobility:** Move component, including execution state
  - Migration: move entire object from one machine to the other
  - Cloning: start a clone, and set it in the same execution state.

Migration In Heterogeneous Systems

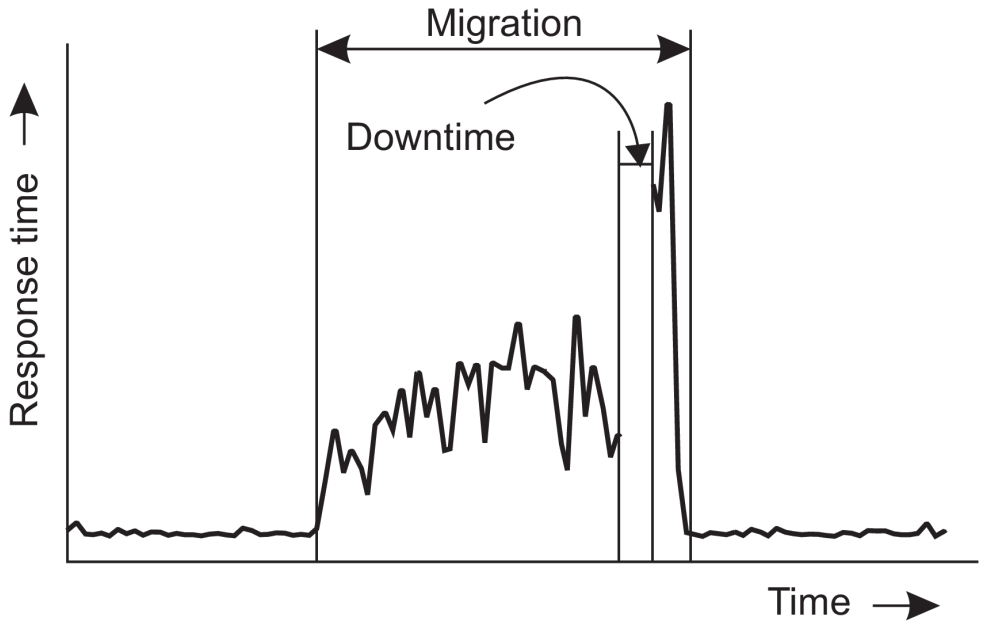
- **Main problem**
  - The target machine may not be suitable to execute the migrated code.
  - The definition of process/thread/processor context is highly dependent on local hardware, operating system and runtime system.
- **Only solution:** abstract machine implemented on different platforms
  - Interpreted languages, effectively having their own VM
  - Virtual machine monitors
- As containers are directly dependent on the underlying operating system, their migration in heterogeneous environments is far from trivial, to simply impractical, just as process migration is.

Migrating a Virtual Machine

- Migrating images: three alternatives
  1. Pushing memory pages to the new machine and resending the ones that are later modified during the migration process.
  2. Stopping the current virtual machine; migrate memory, and start the new virtual machine.
  3. Letting the new virtual machine pull in new pages as needed: processes start on the new virtual machine immediately and copy memory pages on demand.

Performance of Migrating Virtual Machines

- **Problem:** A complete migration may actually take tens of seconds. We also need to realize that during the migration, a service will be completely unavailable for multiple seconds.
- Measurements regarding response times during VM migration



Chapter Wrap-Up

In the next chapter, we will discuss the rules that communicating processes must adhere to, known as protocols, and concentrate on structuring those protocols in the form of layers. We also take a look at below topics:

- Remote Procedure Call,
- Message-oriented Communication,
- Multicast Communication.